

Cómo editar un endpoint

Proyecto To-Do (Spring Boot + Angular)

Guía técnica interna

July 1, 2026

Abstract

Esta guía explica, paso a paso y en orden, cómo modificar un endpoint de la API cuando hay que añadir o cambiar un campo (por ejemplo, añadir el campo *prioridad* a las tareas). Cubre el recorrido completo: base de datos, entidad, DTOs, servicio, controlador, pruebas y el frontend de Angular. Al final hay una **lista de comprobación** y los **errores más comunes** (y cómo evitarlos).

Contents

1	Arquitectura por capas	1
2	Paso 0: la regla de oro de las migraciones	1
3	Pasos en el backend	2
3.1	1. La migración de base de datos	2
3.2	2. La entidad JPA	2
3.3	3. El enum (si el campo es un conjunto cerrado de valores)	2
3.4	4. Los DTOs (entrada y salida)	3
3.5	5. El servicio	3
3.6	6. El controlador	3
3.7	7. Las pruebas	3
4	Pasos en el frontend (Angular)	3
4.1	1. El modelo TypeScript	3
4.2	2. El formulario (diálogo de alta/edición)	4
4.3	3. La tabla y los filtros del dashboard	4
5	Verificación	4
6	Lista de comprobación	4
7	Errores comunes (y cómo evitarlos)	5

1 Arquitectura por capas

El backend está organizado en capas. Una petición HTTP atraviesa siempre el mismo camino, y por eso una modificación suele tocar *todas* las capas de forma coherente:

Cliente (Angular) → Controller → Service → Repository (JPA) → Base de datos
(PostgreSQL)

- **Entidad** (`Task.java`): mapea la tabla a un objeto Java (JPA/Hibernate).
- **DTOs** (`TaskRequest`, `TaskResponse`): datos que *entran* y *salen* de la API. Nunca se expone la entidad directamente.
- **Service** (`TaskService`): lógica de negocio y transacciones.
- **Controller** (`TaskController`): expone las rutas HTTP y valida la entrada.
- **Migración Flyway**: crea/versiona el esquema de la base de datos.

Regla clave: la entidad, los DTOs y el esquema de la base de datos deben describir *los mismos* campos. Si añades un campo a la entidad pero no a la tabla (o al revés), Hibernate falla al arrancar con `ddl-auto: validate`.

2 Paso 0: la regla de oro de las migraciones

Nunca edites una migración de Flyway que ya se aplicó. Crea siempre una migración *nueva* con el siguiente número de versión.

Flyway guarda un *checksum* (huella) de cada script en la tabla `flyway_schema_history`. Si modificas un archivo ya aplicado, el checksum local deja de coincidir con el guardado y la aplicación **no arranca**, con un error como:

```
FlywayValidateException: Migration checksum mismatch for migration version 2
-> Applied to database : -509265412
-> Resolved locally : 1048635791
```

Por eso, para añadir la columna `priority` a una tabla que *ya existe*, lo correcto es crear `V3__add_priority_to_tasks.sql`:

```
-- Añade la columna de prioridad a una tabla ya existente.
-- Se usa un DEFAULT temporal para no romper las filas ya guardadas
-- (la columna es NOT NULL); luego se puede quitar el DEFAULT.
ALTER TABLE tasks
  ADD COLUMN priority VARCHAR(20) NOT NULL DEFAULT 'MEDIA';

ALTER TABLE tasks
  ALTER COLUMN priority DROP DEFAULT;
```

Listing 1: `backend/src/main/resources/db/migration/V3__add_priority_to_tasks.sql`

Si *ya* editaste una migración aplicada y el proyecto está roto, tienes dos opciones para recuperarte en un entorno de desarrollo:

1. Ejecutar `flyway repair` para re-sincronizar el checksum (no modifica el esquema, sólo el historial), o
2. Recrear la base de datos desde cero para que Flyway vuelva a aplicar todas las migraciones. Sólo válido si puedes perder los datos.

3 Pasos en el backend

3.1 1. La migración de base de datos

Crea la migración nueva (ver Paso 0). Comprueba que el tipo y las restricciones (`NOT NULL`, longitud) coincidan con lo que declararás en la entidad.

3.2 2. La entidad JPA

Añade el atributo, su `getter/setter` y (si aplica) al constructor de conveniencia. Un `enum` se guarda como texto con `@Enumerated`:

```
/** Prioridad de la tarea. Se guarda como texto (BAJA / MEDIA / ALTA). */
@Enumerated(EnumType.STRING)
@Column(name = "priority", nullable = false, length = 20)
private TaskPriority priority;

public TaskPriority getPriority() { return priority; }
public void setPriority(TaskPriority p) { this.priority = p; }
```

Listing 2: backend/.../task/Task.java

3.3 3. El enum (si el campo es un conjunto cerrado de valores)

```
public enum TaskPriority {
    BAJA,
    MEDIA, // OJO: el valor real es MEDIA (no "NORMAL")
    ALTA
}
```

Listing 3: backend/.../task/TaskPriority.java

Consistencia: el `enum` es la *fuentes de verdad* de los valores. El frontend y los comentarios del SQL deben usar exactamente los mismos.

3.4 4. Los DTOs (entrada y salida)

En `TaskRequest` añade el campo con su validación; en `TaskResponse` añádelo también y mapéalo desde la entidad.

```
// --- TaskRequest: lo que envía el cliente ---
@NotNull(message = "La prioridad es obligatoria")
TaskPriority priority,

// --- TaskResponse: lo que devuelve la API ---
// (añadir el campo al record y al método fromEntity)
task.getPriority(),
```

Listing 4: TaskRequest.java (entrada) y TaskResponse.java (salida)

3.5 5. El servicio

Propaga el campo tanto al *crear* como al *actualizar*:

```
// create(...)
Task task = new Task(request.title(), request.description(),
                    request.status(), request.priority(),
                    request.dueDate(), owner);

// update(...) -> dirty checking de JPA hace el UPDATE
task.setPriority(request.priority());
```

Listing 5: TaskService.java

3.6 6. El controlador

En este proyecto el `TaskController` recibe y devuelve DTOs, así que normalmente **no hay que cambiar nada**: al ampliar los DTOs, el nuevo campo viaja solo. Sólo se toca si cambias la *firma* del endpoint (ruta, método HTTP, parámetros).

3.7 7. Las pruebas

Actualiza los tests para que el nuevo campo forme parte de los datos de prueba y verifica que se refleja en la respuesta:

```
TaskRequest request = new TaskRequest(
    "Comprar pan", "Integral", TaskStatus.PENDIENTE,
    TaskPriority.MEDIA, LocalDate.now());
...
assertThat(result.priority()).isEqualTo(TaskPriority.MEDIA);
```

Listing 6: TaskServiceTest.java

4 Pasos en el frontend (Angular)

4.1 1. El modelo TypeScript

Refleja los mismos tipos que el backend en `core/models/task.model.ts`:

```
export type TaskPriority = 'BAJA' | 'MEDIA' | 'ALTA';

// Añadir 'priority' a las interfaces Task y TaskRequest, y una lista
// reutilizable de opciones con etiqueta legible:
export const TASK_PRIORITY_OPTIONS: { value: TaskPriority; label: string }[] = [
    { value: 'BAJA', label: 'Baja' },
    { value: 'MEDIA', label: 'Media' },
    { value: 'ALTA', label: 'Alta' },
];
```

Listing 7: task.model.ts (el tipo debe coincidir con el enum del backend)

4.2 2. El formulario (diálogo de alta/edición)

Añade el control al `FormGroup`, la opción al `<mat-select>` y el campo tanto al precargar (edición) como al construir el `TaskRequest`.

4.3 3. La tabla y los filtros del dashboard

Cuidado con dos trampas frecuentes de Angular Material:

1. Si añades una columna a `displayedColumns`, *debes* crear su `<ng-container matColumnDef="...">` en la plantilla. Si no, la tabla falla en tiempo de ejecución con "Could not find column with id ..." (no lo detecta la compilación).
2. Un filtro no funciona sólo con pintar el `<mat-select>`: hay que *suscribirse* a sus `valueChanges`, usarlo dentro de la función de filtrado y resetearlo al limpiar filtros.

5 Verificación

```
# Backend: compilar + pruebas
cd backend && ./mvnw test
# Backend: arrancar (aplica migraciones Flyway al inicio)
./mvnw spring-boot:run

# Frontend: compilar (detecta errores de tipos y de plantilla)
cd frontend && npm run build
# Frontend: servir en desarrollo
npm start
```

6 Lista de comprobación

- ☐ Migración **nueva** (no editar una ya aplicada).
- ☐ Entidad: campo + getter/setter + constructor.
- ☐ Enum creado (si aplica) y usado como fuente de verdad.
- ☐ DTOs de entrada y salida actualizados (con validación).
- ☐ Servicio: crear *y* actualizar.
- ☐ Pruebas actualizadas y en verde.
- ☐ Modelo TS con el mismo tipo que el backend.
- ☐ Formulario: control + opción + carga + envío.
- ☐ Tabla: `displayedColumns` y `matColumnDef`.
- ☐ Filtro suscrito, aplicado y reseteado.
- ☐ `./mvnw test` y `npm run build` sin errores.

7 Errores comunes (y cómo evitarlos)

Checksum de Flyway.

Editar una migración ya aplicada. *Solución:* crear una migración nueva; nunca tocar las aplicadas.

Hibernate validate falla al arrancar.

La entidad y la tabla no coinciden. *Solución:* que la columna (tipo, NOT NULL, longitud) case con la anotación `@Column`.

Could not find column with id ...

Añadir la columna a `displayedColumns` sin su `matColumnDef`. *Solución:* crear el `<ng-container matColumnDef>` correspondiente.

El filtro/menú "no hace nada".

Un control declarado pero no conectado a la lógica (sin `valueChanges`), o un `<mat-sidenav>` desligado de la señal que lo controla. *Solución:* mantener el `[opened]`/`valueChanges` enlazados con el estado del componente.

Valores desalineados.

El enum dice `MEDIA` pero un comentario o el frontend dicen `NORMAL`. *Solución:* el enum manda; todo lo demás debe copiar sus valores exactos.